

Что изучать

Карта тем по проекту Lore Assistant — RAG-чат на FastAPI, PostgreSQL и Gemini

Как пользоваться

Темы идут от простого к сложному. По каждой: суть в двух-трёх предложениях, точка в нашем коде, где её можно потрогать, и английский запрос для поиска — почти вся литература по AI на английском. Метка **НЕ ИСПОЛЬЗУЕМ** значит, что приём в проекте не реализован, но знать его стоит: это следующий шаг для роста.

ЕСТЬ

реализовано в проекте

НЕ ИСПОЛЬЗУЕМ

следующий шаг

ПРОБЕЛ

стоило бы

починить

1 Python и асинхронность

Асинхронное программирование [async / await, asyncio](#) **ЕСТЬ**

Пока код ждёт ответа от сети или базы, процессор простаивает. Async позволяет в это время заняться другим запросом. Весь наш проект асинхронный: и обращения к Gemini, и запросы к Postgres.

У нас: `async def execute(...)` в `app/use_cases/chat.py`

Искать: "python asyncio event loop", "async await explained"

Асинхронные генераторы [async generators, yield](#) **ЕСТЬ**

Функция, которая отдаёт значения по одному, не собирая всё в память. Именно так работает стриминг ответа: модель отдаёт кусок текста — мы сразу шлём его в браузер, не дожидаясь конца.

У нас: `AsyncGenerator[str, None]` в `generate_answer_stream`

Искать: "python async generator", "yield vs return"

Протоколы и утиная типизация [typing.Protocol, structural typing](#) **ЕСТЬ**

Способ описать интерфейс без наследования: «мне нужен объект с методом `search_similar`, а чей он класс — неважно». На этом держатся наши порты.

У нас: `app/domain/ports.py`

Искать: "python typing Protocol", "structural subtyping"

2 Веб и API

FastAPI и внедрение зависимостей [dependency injection, Depends](#) **ЕСТЬ**

Функция-обработчик не создаёт себе базу и клиента модели, а получает их снаружи. Это позволяет подменить их в тестах и не плодить связность.

У нас: `app/api/dependencies.py`

Искать: "fastapi dependency injection", "Depends yield"

Валидация схемами [pydantic, request validation](#) **ЕСТЬ**

Описываешь, как выглядит корректный запрос — фреймворк сам отсекает мусор до того, как он дойдёт до логики. Неизвестный режим ответа у нас получает 422 автоматически.

У нас: `app/api/schemas.py`

Искать: "pydantic v2 models", "fastapi request validation"

Потоковая отдача ответа [HTTP chunked streaming](#) **ЕСТЬ**

Ответ уходит частями по мере готовности, а не целиком в конце. Поэтому текст начинает появляться сразу. Важная деталь: как только отдан первый байт, статус ответа уже 200 и поменять его на ошибку нельзя.

У нас: `StreamingResponse` в `app/api/router.py`

Искать: "http chunked transfer encoding", "fastapi StreamingResponse"

Server-Sent Events [SSE, EventSource](#) **НЕ ИСПОЛЬЗУЕМ**

Настоящий протокол потоковых событий: каждое сообщение оформляется как `data: ...` и отделяется пустой строкой, браузер читает его через `EventSource`. У нас стоит SSE-заголовок, но формат обычный — сырой текст, который фронт читает через `getReader()`. Стоит понимать разницу.

Искать: "server sent events vs websockets", "EventSource API"

3 База данных

SQLAlchemy 2.0, асинхронный режим [async ORM](#), [AsyncSession](#) **ЕСТЬ**

ORM переводит объекты Python в строки таблиц. В версии 2.0 новый синтаксис: `Mapped`, `mapped_column`, `select()` вместо старого `query()`.

У нас: `app/infrastructure/database/models.py`

Искать: "sqlalchemy 2.0 async", "Mapped mapped_column"

Драйвер и пул соединений [psycopg3](#), [connection pool](#) **ЕСТЬ**

Драйвер — библиотека, которая говорит с Postgres по его протоколу. У нас psycopg 3 (не asyncpg). Пул держит соединения открытыми, чтобы не устанавливать их заново на каждый запрос.

У нас: `create_async_engine` в `connection.py`

Искать: "psycopg3 async", "database connection pooling"

Транзакции и Unit of Work [transaction](#), [commit](#), [rollback](#) **ЕСТЬ**

Группа изменений применяется целиком или не применяется вовсе. У нас сессия выдаётся зависимостью с `yield`: при ошибке — откат, в конце — закрытие. Коммита нет, потому что чат только читает.

У нас: `get_async_session`

Искать: "unit of work pattern", "ACID transactions"

Каскадное удаление [ON DELETE CASCADE](#), [foreign key](#) **ЕСТЬ**

Удалили статью — её чанки и слова словаря уходят следом автоматически, на уровне базы. Благодаря этому перезаливка лора не требует ручной чистки.

У нас: `ForeignKey(..., ondelete="CASCADE")` в моделях

Искать: "postgres foreign key cascade", "referential integrity"

Идемпотентность [idempotency](#), [upsert](#) **ЕСТЬ**

Операцию можно повторить, и результат не изменится. Наши загрузчики находят статью по URL, сносят и заливают заново — поэтому запускать их можно сколько угодно раз.

У нас: `load_lore.py`, `load_human.py`

Искать: "idempotent operations", "postgres ON CONFLICT DO NOTHING"

4 Векторы и эмбединги

Эмбединги [embeddings](#), [vector representation](#) **ЕСТЬ**

Текст превращается в список чисел так, что близкие по смыслу тексты дают близкие списки. Это фундамент семантического поиска: искать можно по смыслу, а не по словам. У нас 768 чисел на кусок текста.

У нас: `generate_embedding` в `gemini_adapter.py`

Искать: "what are text embeddings", "vector representation of text"

Размерность вектора [dimensionality, Matryoshka embeddings](#) ЕСТЬ

Сколько чисел в векторе. Больше — точнее, но дороже по памяти и медленнее. Модель `gemini-embedding-001` по умолчанию даёт 3072, мы урезаем до 768 через `output_dimensionality`.

Искать: "matryoshka representation learning", "embedding dimensionality tradeoff"

Косинусное расстояние [cosine distance vs L2 / euclidean](#) ЕСТЬ

Мера близости по **углу** между векторами, а не по длине. Поэтому длинный и короткий текст об одном и том же считаются близкими. Евклидово расстояние (L2) учитывало бы и длину — для текста это обычно хуже.

У нас: `embedding.cosine_distance(...)` в `postgres_adapter.py`

Искать: "cosine similarity vs euclidean distance", "pgvector operators"

Типы задач для эмбеддингов [task types, asymmetric retrieval](#) ПРОБЕЛ

Документ и поисковый запрос устроены по-разному: документ — развёрнутый текст, запрос — короткий вопрос. Поэтому кодировать их положено разными режимами (`RETRIEVAL_DOCUMENT` и `RETRIEVAL_QUERY`). У нас везде первый — это известный недочёт.

Искать: "asymmetric semantic search", "gemini embedding task types"

Векторная база данных [vector database, pgvector](#) ЕСТЬ

Хранилище, умеющее искать «похожие векторы». Можно взять отдельную базу (Pinecone, Qdrant, Weaviate), а можно расширение к обычному Postgres — мы выбрали второе, чтобы держать текст и векторы в одном месте.

Искать: "pgvector tutorial", "vector database comparison"

5 Векторные индексы

Почему у нас их нет

На `article_chunks.embedding` индекса нет вообще — поиск идёт полным перебором. При десяти кусках текста это *быстрее* любого индекса. Тема становится нужной, когда чанков тысячи.

Приближённый поиск соседей ANN, [approximate nearest neighbor](#) НЕ ИСПОЛЬЗУЕМ

Точный поиск сравнивает запрос с каждым вектором — при миллионе записей это недопустимо долго. Приближённый жертвует малой долей точности ради огромного выигрыша в скорости. Ключевой компромисс — recall против latency.

Искать: "approximate nearest neighbor search", "recall vs latency"

IVFFlat [inverted file index](#) НЕ ИСПОЛЬЗУЕМ

Векторы заранее разбиваются на кластеры. При поиске проверяются только ближайшие кластеры, а не всё. Строится быстро и занимает мало места, но требует, чтобы данные уже были загружены — на пустой таблице кластеры не построить.

Искать: "pgvector ivfflat lists", "inverted file index vector search"

HNSW [hierarchical navigable small world](#) НЕ ИСПОЛЬЗУЕМ

Многослойный граф: верхние слои дают грубые прыжки к нужной области, нижние — точную настройку. Обычно быстрее и точнее IVFFlat, но дольше строится и занимает больше памяти. Сегодня выбор по умолчанию для большинства задач.

Искать: "HNSW algorithm explained", "pgvector hnsw m ef_construction"

6 RAG — основы

Генерация с опорой на поиск RAG, [retrieval-augmented generation](#) ЕСТЬ

Вместо того чтобы полагаться на память модели, мы сначала ищем нужные куски текста в своей базе, а потом кладём их в промт как контекст. Так модель отвечает по вашим данным, а не по интернету, и вы контролируете источник.

У нас: вся `ChatUseCase.execute`

Искать: "RAG pipeline explained", "retrieval augmented generation"

Разбиение на куски [chunking](#) ЕСТЬ

Длинный текст режется на части, каждая получает свой вектор. Слишком крупные куски размывают смысл, слишком мелкие теряют контекст. У нас самый простой вариант — абзац как кусок.

У нас: `raw_text.split('\n')` в `load_lore.py`

Искать: "chunking strategies RAG", "optimal chunk size"

Фильтрация по метаданным [metadata pre-filtering](#) ЕСТЬ

Сначала отсекаем лишнее обычным условием, потом ищем по векторам среди оставшегося. У нас так работает вселенная: сравнивать «Альтаира» с чанками про другого персонажа незачем — и быстрее, и точнее.

У нас: `.where(ArticleChunk.universe == universe)`

Искать: "metadata filtering vector search", "pre-filter vs post-filter"

Системный промт и ограничения [system prompt, negative constraints](#) ЕСТЬ

Инструкция модели, как себя вести. Важно, чтобы правила не противоречили друг другу: у нас «рассказывай литературно» воевало с «ничего не выдумывай», и модель дорисовывала детали. Пришлось развести явно — украшать можно язык, но не факты.

У нас: `_STORYTELLER` и `_ARCHIVIST` в `gemini_adapter.py`

Искать: "system prompt engineering", "prompt constraints hallucination"

Температура [temperature, sampling](#) ЕСТЬ

Насколько модель склонна выбирать менее вероятные слова. 0 — почти детерминированно, выше — разнообразнее и рискованнее. У нас 0.5 для рассказчика, 0.1 для архивариуса, 0.0 для переписывания запроса.

Искать: "llm temperature top_p explained"

Галлюцинации и заземление [hallucination, grounding](#) ЕСТЬ

Модель может уверенно выдумать факт. Заземление — заставить её опираться только на переданный контекст. Полностью проблему это не снимает: чем выше температура и чем хуже найденный контекст, тем выше риск.

Искать: "llm hallucination grounding", "faithfulness RAG"

7 RAG — продвинутое

Переписывание запроса `query rewriting` **ЕСТЬ**

Прежде чем искать, запрос улучшают: исправляют опечатки, раскрывают местоимения по истории диалога. «А что он усовершенствовал?» превращается в «Что Альтаир усовершенствовал?» — иначе поиск не найдёт ничего.

У нас: `rewrite_query`, вызывается условно

Искать: "query rewriting RAG", "conversational query reformulation"

Порог релевантности `similarity threshold` **ПРОБЕЛ**

Отбрасывать найденное, если оно слишком далеко от запроса. Без порога система всегда возвращает top-N, даже когда ничего подходящего нет, — и модель начинает сочинять по нерелевантному контексту. У нас порога нет.

Искать: "similarity threshold retrieval", "when to return no results"

Перекрытие кусков `chunk overlap` **НЕ ИСПОЛЬЗУЕМ**

Соседние куски делают частично пересекающимися, чтобы мысль, разорванная границей, не потерялась. Стоит дешевле почти всех остальных улучшений.

Искать: "chunk overlap RAG", "sliding window chunking"

Переранжирование `reranking, cross-encoder` **НЕ ИСПОЛЬЗУЕМ**

Двухступенчатый поиск: сначала быстро достаём 50 кандидатов векторами, потом отдельной моделью точно оцениваем каждого и оставляем лучшие 5. Обычно даёт самый заметный прирост качества.

Искать: "cross-encoder reranking", "two-stage retrieval"

Гибридный поиск `hybrid search, BM25, RRF` **НЕ ИСПОЛЬЗУЕМ**

Векторы плохо ищут точные совпадения — артикулы, имена, термины. Полнотекстовый поиск наоборот. Их объединяют, а результаты сливают алгоритмом Reciprocal Rank Fusion.

Искать: "hybrid search BM25 vector", "reciprocal rank fusion"

HyDE `hypothetical document embeddings` **НЕ ИСПОЛЬЗУЕМ**

Приём: попросить модель придумать *гипотетический ответ* на вопрос и искать по вектору этого ответа. Выдуманный ответ по форме ближе к документам, чем вопрос, поэтому поиск часто срабатывает лучше.

Искать: "HyDE hypothetical document embeddings"

Оценка качества RAG `RAG evaluation, RAGAS` **НЕ ИСПОЛЬЗУЕМ**

Как понять, что изменение сделало лучше, а не хуже? Метрики: попал ли нужный кусок в выдачу, опирается ли ответ на контекст, отвечает ли он на вопрос. Без этого любая доработка — гадание.

Искать: "RAGAS metrics", "context precision recall faithfulness"

8 Текстовый поиск в PostgreSQL

Триграммы [trigram, pg_trgm](#) **ЕСТЬ**

Слово режется на трёхбуквенные кусочки, похожесть — доля общих кусочков. Так ловятся опечатки: «Альтарир» и «Альтаир» дают 0.545, а разные имена — ровно 0. Регистр и знаки препинания нормализуются попутно.

У нас: `correct_typos` в `postgres_adapter.py`

Искать: "postgres pg_trgm similarity", "fuzzy string matching"

GiST против GIN [GiST vs GIN index, KNN search](#) **ЕСТЬ**

Два вида индексов для триграмм. GIN быстрее для фильтра «похоже ли», но **не умеет** сортировку по близости (`ORDER BY term <-> 'слово'`) — запрос молча вырождается в полный перебор. Для поиска ближайшего нужен GiST. Ловушка, на которую легко попасться.

У нас: `ix_lore_terms_term_trgm`, тип `gist`

Искать: "postgres GiST vs GIN trigram", "KNN ORDER BY distance index"

Расстояние Левенштейна [Levenshtein distance, edit distance](#) **НЕ ИСПОЛЬЗУЕМ**

Сколько правок (вставить, удалить, заменить букву) нужно, чтобы получить из одного слова другое. Для «Альтарир» → «Альтаир» это 1. Живёт в расширении `fuzzystrmatch`, которое у нас не установлено.

Искать: "levenshtein distance algorithm", "postgres fuzzystrmatch"

Фонетические алгоритмы [Soundex, Metaphone](#) **НЕ ИСПОЛЬЗУЕМ**

Сравнивают слова по звучанию. Важное предупреждение: в PostgreSQL они рассчитаны на латиницу и **на кириллице возвращают пустую строку для любого слова** — то есть формально все слова «совпадают» со всеми. Проверено. Для русского из `fuzzystrmatch` годится только Левенштейн.

Искать: "soundex limitations non-english", "phonetic matching algorithms"

План запроса [EXPLAIN, query plan, Seq Scan](#) **ЕСТЬ**

Команда `EXPLAIN` показывает, как база собирается выполнять запрос и использует ли индекс. Единственный способ убедиться, что индекс реально работает, а не лежит мёртвым грузом.

Искать: "postgres EXPLAIN ANALYZE", "reading query plans"

9 Архитектура и паттерны

Гексагональная архитектура [hexagonal architecture, ports and adapters](#) **ЕСТЬ**

Бизнес-логика в центре и ничего не знает о внешнем мире. Всё остальное — база, модель, HTTP — подключается через интерфейсы. Заменить Postgres на другое хранилище можно, не тронув логику.

У нас: вся структура `app/`

Искать: "hexagonal architecture", "ports and adapters pattern"

Правило направления зависимостей [dependency rule, clean architecture](#) **ЕСТЬ**

Внешние слои знают о внутренних, но не наоборот. Проверяется простым grep: если `use_cases/` импортирует `infrastructure/` — правило нарушено.

Искать: "clean architecture dependency rule", "Uncle Bob clean architecture"

Порт и адаптер [port, adapter, interface segregation](#) **ЕСТЬ**

Порт — «что нужно уметь», адаптер — «как это делается для конкретной технологии». Полезно следить, чтобы порт не разрастался: наш `IVectorStore` уже включает исправление опечаток, что к «векторному хранилищу» отношения не имеет.

Искать: "interface segregation principle", "SOLID principles"

Композиционный корень [composition root](#) **ЕСТЬ**

Единственное место, где создаются конкретные реализации и связываются друг с другом. У нас это `api/dependencies.py` — только он знает одновременно и про Postgres, и про Gemini.

Искать: "composition root pattern", "dependency injection container"

10 Фронтенд

Чтение потока в браузере [Streams API, ReadableStream](#) **ЕСТЬ**

Обычный `fetch` ждёт весь ответ. Через `res.body.getReader()` можно читать байты по мере поступления и сразу показывать текст. `TextDecoder` с флагом `stream: true` нужен, чтобы не разорвать многобайтовый символ между кусками — для кириллицы это критично.

Искать: "fetch streaming response getReader", "TextDecoder stream option"

Анимация по времени, а не по кадрам [requestAnimationFrame, delta time](#) **ЕСТЬ**

Если считать «столько-то символов за кадр», на 120-герцевом мониторе всё побежит вдвое быстрее. Правильно — считать от реально прошедшего времени.

Искать: "requestAnimationFrame delta time", "frame rate independent animation"

Анимация градиента [CSS @property, conic-gradient](#) ЕСТЬ

По умолчанию браузер не умеет плавно менять угол градиента — он для него просто строка.

`@property` объявляет переменную как угол, и она становится анимируемой. Так сделан бегущий луч по рамке ответа.

Искать: "CSS @property houdini", "animated gradient border"

Маска-кольцо [mask-composite, gradient border](#) ЕСТЬ

Приём: рисуем градиент на весь элемент, потом маской вырезаем середину — остаётся только полоска по краю. Стандартный способ сделать градиентную рамку, которую `border` не умеет.

Искать: "css mask-composite exclude", "gradient border trick"

Контраст и доступность [WCAG contrast ratio, a11y](#) ЕСТЬ

Отношение яркостей текста и фона. Норма AA — 4.5:1 для обычного текста. Это измеримо, а не вкусовщина: серый `slate-500` на нашем фоне давал 3.98:1 (провал), `slate-400` — 7.38:1.

Искать: "WCAG contrast ratio calculator", "web accessibility basics"

11 Куда расти дальше

Порядок не случайный — от самого выгодного к менее срочному.

1. **Метрики качества.** Без них любое улучшение — гадание. Сначала научиться измерять, потом менять.
2. **Порог релевантности.** Самая дешёвая правка с прямым эффектом на выдумки: пусть система честно отвечает «не нашёл», когда не нашла.
3. **Правильный `task_type` для запроса.** Одна строка кода, перезаливка базы не нужна.
4. **Перекрытие кусков.** Тоже дёшево, заметно помогает на связных текстах.
5. **Векторный индекс HNSW.** Когда чанков станет хотя бы несколько тысяч. Раньше — только замедление.
6. **Переранжирование.** Самый большой прирост качества, но нужна вторая модель и лишняя задержка.
7. **Гибридный поиск.** Когда появятся точные термины, имена и артикулы, которые векторы ищут плохо.

Главный вывод не про технологии

Документ-инвентаризация этого же проекта, составленный языковой моделью, содержал семь фактических ошибок: неверный драйвер базы, неверный SDK, несуществующие расширения PostgreSQL, неверная температура и два приёма RAG, которых в коде нет вовсе. Текст был складным и убедительным. Проверяйте по коду — особенно то, что звучит уверенно.